

SMC CLINT — Test Plan

SMC CLINT — Detailed Test Plan

Document: 03_CLINT_Test_Plan.md **Module under test (MUT):** smc::clint (peripherals/clint/)

Reference test benches: - peripherals/clint/test/clint_tb.cpp — 22 deterministic test groups
(tick_period_ns = 0) - peripherals/clint/test/clint_tick_tb.cpp — 3 auto-tick test groups
(tick_period_ns = 50) **Status:** Active — 22 + 3 unit-test groups pass on SystemC 3.0.2 / Apple Clang 17

Companion docs: - 01_CLINT_Specification.md — externally-observable behaviour -
02_CLINT_LowLevel_Design.md — implementation contract

Contents

1. Objectives
 2. Scope (in / out)
 3. Verification strategy
 4. Environment
 5. Test bench architecture
 6. Stimulus & checking primitives
 7. Feature → test traceability matrix
 8. Detailed test cases (TC-1 ... TC-22, TA-1 ... TA-3)
 9. Coverage plan
 10. Negative-test catalogue
 11. Regression workflow
 12. Pass / fail criteria & exit conditions
 13. Future tests & open work
 14. Test results — current baseline
-

1. Objectives

The CLINT test plan must demonstrate that the SystemC model:

1. **Conforms** to the RISC-V Privileged Architecture (machine-mode timer + software interrupt) and the OCAH `clint.rdl` register layout.
2. **Implements correctly** every flow described in 01_CLINT_Specification.md §6 (MTIME tick, MTIP comparator, MSIP propagation, per-hart isolation, reset).
3. **Honours** the TLM-2.0 contract documented in 01_CLINT_Specification.md §10 (alignment, width, error responses, debug back door).
4. **Is robust** against negative stimulus (misaligned, oversize, out-of-window, wrong access width, bad commands).
5. **Is integration-ready** — the unmodified firmware driver (`fw/smc/common/drivers/riscv_clint0.c`) runs against it through the full fabric path, including the rollover-safe MTIME read sequence and the “high-FF /

low / high” MTIMECMP set sequence.

6. **Is CCI 1.0 compliant** — parameters are discoverable, mutable (where declared), immutable (where declared), and have correct metadata.

2. Scope (in / out)

2.1 In scope

- Self-checking SystemC unit tests for every register and every observable behavioural rule.
- Test-bench-driven stimulus through `b_transport` (the production access path), in both 4-byte and 8-byte forms.
- Back-door inspection through `dbg_*` and `transport_dbg`.
- Negative path testing (misaligned, oversize, undersize, out-of-window, wrong width per register).
- Reset behaviour, including from a non-trivial state.
- CCI parameter discovery, mutation, and immutability enforcement.

2.2 Out of scope (covered by other plans)

Aspect	Owner / document
Cycle-accurate timing of MSIP/MTIP delivery	RTL verification (Chipyard)
Access-control / <code>axi_filter</code> enforcement	<code>axi_filter</code> test plan
Multi-master arbitration on the SMC fabric	<code>smc_fabric</code> test plan
<code>riscv_clint0.c</code> driver-level scenarios	SMC firmware test plan
Whisper ↔ Rocket commit-log diff	Co-simulation test plan (future)
Power / DFT / MBIST	RTL DV plan
Parity / lockstep safety extensions	Functional-safety test plan (future)

3. Verification strategy

3.1 Tier model

Tier	Goal	Where
Unit (this plan)	Per-register R/W, per-flow correctness	<code>clint_tb.cpp</code>
Integration	Fabric → filter → CLINT → CPU cluster	<code>smc_top</code> integration test (future)
System	Firmware running real CLINT driver under Whisper	<code>smc_top</code> boot test (future)
Co-simulation	Whisper vs Rocket commit-log diff with timer/IPI events	future

3.2 Methods

Method	Purpose
Directed tests	Trigger a specific spec requirement and check the observable response.
Back-door assertions (<code>dbg_*</code>)	Inspect internal state with no perturbation.
Deterministic time control	Disable auto-tick (<code>tick_period_ns = 0</code>) and walk MTIME via <code>dbg_set_mtime</code> .
Firmware-pattern replay	Mirror the exact <code>riscv_clint0.c</code> access sequences (32-bit halves, hi/lo/hi).
Negative stimulus	Confirm the bus-error contract holds for misaligned / oversize / wrong width.
Reset replay	Drive non-trivial state, then assert reset, then re-check from scratch.
CCI introspection	Discover params via the broker, verify metadata, mutate / fail to mutate.
Two-phase settling	After every state mutation, run two delta cycles to let <code>recompute_event_</code> propagate before checking outputs.

3.3 Determinism guarantees

The test bench is fully deterministic:

- No random stimulus.
- `tick_period_ns = 0` is preset via CCI so MTIME never advances on its own — the TB walks MTIME explicitly via `dbg_set_mtime` to produce bit-identical runs from machine to machine.
- No wall-clock dependence; settles via `wait(SC_ZERO_TIME)` instead of real time.
- Single-threaded SystemC scheduler, single TLM initiator.

A failing test is a hard error — no retries, no flaky regression flags.

4. Environment

Component	Required version
Accellera SystemC	≥ 2.3.4 (3.0.2 verified)
Accellera SystemC CCI	1.0.x (1.0.2 verified)
C++ compiler	C++17 — Apple Clang 17 / Clang 10+ / GCC 9+
CMake	≥ 3.16
OS (host)	macOS / Linux
Optional	<code>ctest</code> for CI integration (already wired)

Reproducible commands:

```
SYSTEMC_HOME=/opt/homebrew/opt/systemc \
CCI_HOME=/Users/pdroy/cci \
  cmake -S peripherals/clint -B peripherals/clint/build -DCMAKE_BUILD_TYPE=Release
cmake --build peripherals/clint/build -j
ctest --test-dir peripherals/clint/build --output-on-failure
```

Or, equivalently:

```

peripherals/clint/run_tests.sh           # configure (if needed) + build + run
peripherals/clint/run_tests.sh --clean   # wipe build/ first
peripherals/clint/run_tests.sh --ctest   # run via ctest with PASS/FAIL regex
peripherals/clint/run_tests.sh --asan    # AddressSanitizer build + run
peripherals/clint/run_tests.sh --coverage # coverage build + HTML report

```

The CMake CTest entry asserts on the regex:

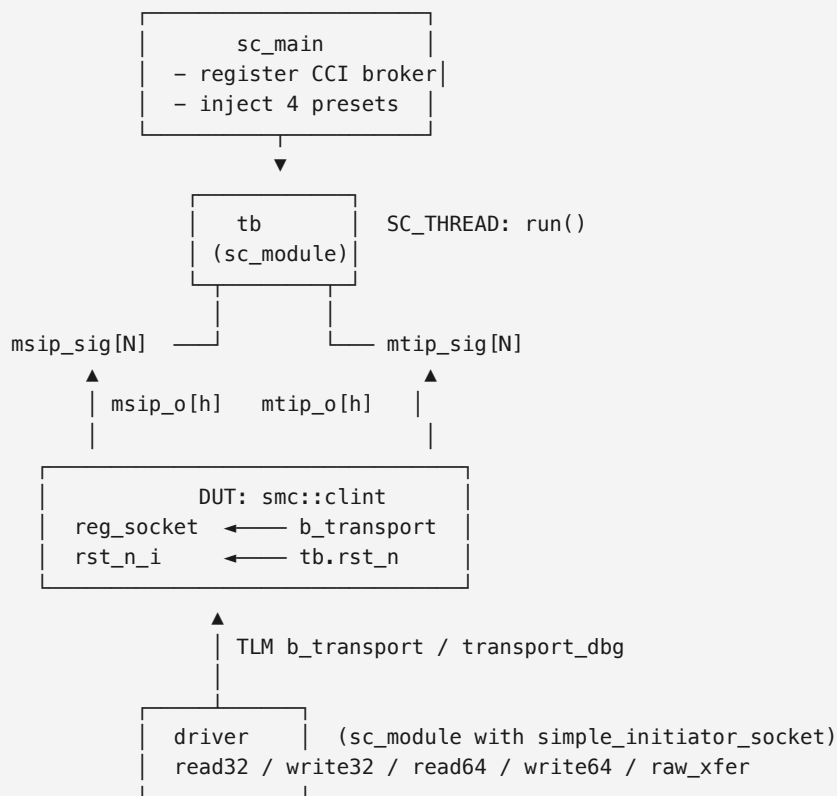
```

PASS_REGULAR_EXPRESSION "ALL TESTS PASSED"
FAIL_REGULAR_EXPRESSION "FAIL"

```

so any test bench run that does not print `ALL TESTS PASSED` is a hard CI failure.

5. Test bench architecture



`tb` instantiates:

- `dut` — the CLINT under test, with CCI presets (`num_harts = 2`, `tick_period_ns = 0`, `access_delay_ns = 5`).
- `drv` — a TLM initiator that exposes `read32 / write32 / read64 / write64 / raw_xfer`.
- `rst_n` and `sc_vector<sc_signal<bool>>` arrays for `msip_o` / `mtip_o`.

The `run()` thread executes the test sequence; on completion it calls `sc_stop()` and returns the failure count to the OS.

6. Stimulus & checking primitives

Primitive	Purpose
<code>EXPECT_EQ(a, b)</code>	Hard equality check; logs <code>FAIL</code> on mismatch and increments <code>g_failures</code> .
<code>EXPECT_TRUE(c)</code>	Boolean-truth check; same failure semantics as <code>EXPECT_EQ</code> .
<code>pulse_reset()</code>	Drives <code>rst_n</code> low for 20 ns, then high for 20 ns, then yields enough delta cycles for state to settle.
<code>settle()</code>	Two <code>wait(SC_ZERO_TIME)</code> calls — enough for <code>b_transport</code> → <code>recompute_event</code> → <code>output_method</code> to propagate to <code>msip_o</code> / <code>mtip_o</code> .
<code>set_mtime_and_settle(v)</code>	Calls <code>dbg_set_mtime(v)</code> , settles, asserts <code>dbg_mtime() == v</code> .
<code>drv.read32(addr)</code>	TLM 32-bit read; auto-fails on non- <code>TLM_OK_RESPONSE</code> .
<code>drv.write32(addr, value)</code>	TLM 32-bit write; auto-fails on non- <code>TLM_OK_RESPONSE</code> .
<code>drv.read64(addr)</code>	TLM 64-bit read; auto-fails on non- <code>TLM_OK_RESPONSE</code> .
<code>drv.write64(addr, value)</code>	TLM 64-bit write; auto-fails on non- <code>TLM_OK_RESPONSE</code> .
<code>drv.raw_xfer(cmd, addr, len, data)</code>	Returns the raw <code>tlm_response_status</code> for negative-path tests.
<code>fw_set_mtimecmp(drv, h, v)</code>	Mirrors the firmware “hi=FFFFFFFF, lo, hi” sequence.
<code>fw_read_mtime(drv)</code>	Mirrors the firmware rollover-safe MTIME read sequence.

Address arithmetic helpers:

```

msip_addr(h)      = MSIP_BASE + MSIP_STRIDE * h
mtimecmp_addr(h)  = MTIMECMP_BASE + MTIMECMP_STRIDE * h
mtime_addr()      = MTIME_OFFSET // 0xBFF8

```

7. Feature → test traceability matrix

#	Specification requirement (01_CLINT_Specification.md)	Verified by
F1	Reset clears MTIME and MSIP[h]	TC-1, TC-9
F2	Reset sets MTIMECMP[h] to 0xFFFF...F (model deviation; deterministic MTIP=0)	TC-1
F3	MSIP only bit[0] writable; bits[31:1] RAZ/WI	TC-2
F4	msip_o[h] tracks MSIP[h].bit[0]	TC-2
F5	Per-hart MSIP isolation	TC-3
F6	MTIME 64-bit single-access R/W	TC-4
F7	MTIME 32-bit half-word R/W (low at +0, high at +4)	TC-5
F8	Firmware rollover-safe MTIME read sequence supported	TC-6
F9	MTIMECMP 64-bit single-access R/W	TC-7
F10	Firmware “high-FF / low / high” MTIMECMP set sequence supported	TC-8
F11	mtip_o[h] = (MTIME ≥ MTIMECMP[h]) — boundary at -1 / == / +1	TC-9
F12	MTIP cleared by writing MTIMECMP > MTIME	TC-10
F13	Per-hart MTIMECMP independence	TC-11
F14	MSIP and MTIP independent per hart	TC-12
F15	MTIME advance via TB-controlled writes / dbg_set_mtime	TC-13
F16	transport_dbg back-door read/write, no side effects	TC-14
F17	Out-of-window access → TLM_ADDRESS_ERROR_RESPONSE	TC-15
F18	Misaligned access → TLM_BURST_ERROR_RESPONSE	TC-15
F19	Wrong access width (e.g. 8 B on MSIP) → TLM_BURST_ERROR_RESPONSE	TC-15
F20	CCI parameter discovery by hierarchical name	TC-16
F21	CCI metadata round-trip (descriptions, units)	TC-16
F22	Mutable CCI parameter mutation observable in next transaction	TC-16
F23	Immutable CCI parameter rejects post-elaboration writes	TC-16
F24	dump_state() produces human-readable snapshot	TC-17
F25	DMI not granted (no get_direct_mem_ptr callback)	Implicit (callback never registered)
F26	MTIMECMP 32-bit half-word READ (low at +0, high at +4)	TC-18
F27	In-window reserved address is RAZ/WI (silent OK on R/W)	TC-19
F28	streaming_width ≠ data_length → TLM_BURST_ERROR_RESPONSE	TC-20
F29	Non-null byte_enable_ptr → TLM_BYTE_ENABLE_ERROR_RESPONSE	TC-21
F30	TLM_IGNORE_COMMAND → TLM_COMMAND_ERROR_RESPONSE	TC-21
F31	transport_dbg rejects invalid args (length / alignment / window) → returns 0	TC-22
F32	Auto-tick advances MTIME and drives MTIP; reset cancels and re-arms the tick	TA-1, TA-2, TA-3

Spec features F1–F24 and F26–F32 are exercised by the active test suite; F25 is covered structurally (the callback is never registered) but not by an explicit assertion — flagged in §13 as a candidate.

8. Detailed test cases (TC-1 ... TC-22, TA-1 ... TA-3)

Each test case lists: *Spec ref* → *Setup* → *Stimulus* → *Expected* → *Notes*. The TC-* cases run in `clint_tb.cpp` (deterministic, `tick_period_ns = 0`); the TA-* cases run in `clint_tick_tb.cpp` (auto-tick, `tick_period_ns = 50 ns`).

TC-1. Reset clears MTIME / MSIP / MTIP; MTIMECMP=max

- **Spec:** F1, F2
- **Setup:** Default `clint_cfg`; CCI presets `num_harts=2`, `tick_period_ns=0`.
- **Stimulus:** Drive `rst_n` low for 20 ns, then high; allow settle.
- **Expected:**
 - `dbg_mtime() == 0`.
 - For every `h`: `dbg_msip(h) == 0`, `dbg_mtimecmp(h) == 0xFFFF_FFFF_FFFF_FFFF`, `dbg_mtip(h) == false`, `msip_sig[h].read() == false`, `mtip_sig[h].read() == false`.

TC-2. MSIP R/W truncates to bit[0]; msip_o tracks register

- **Spec:** F3, F4
- **Setup:** After reset.
- **Stimulus** (per hart):
 - `write32(msip_addr(h), 0xFFFFFFFF)` — should truncate to bit[0].
 - `read32(msip_addr(h))` → expect 1.
 - settle, expect `msip_sig[h] == true`.
 - `write32(msip_addr(h), 0)` → read returns 0, `msip_sig[h] == false`.

TC-3. MSIP per-hart isolation

- **Spec:** F5
- **Setup:** After reset.
- **Stimulus:** `write32(msip_addr(0), 1)`; settle.
- **Expected:** `msip_sig[0] == true`; for all `h ≠ 0`, `msip_sig[h] == false`. Then clear `MSIP[0]` and confirm cleanup.

TC-4. MTIME 64-bit R/W

- **Spec:** F6
- **Stimulus:** `write64(mtime_addr(), 0x0123_4567_89AB_CDEF)`, then `read64(mtime_addr())`.
- **Expected:** read returns the same 64-bit pattern; `dbg_mtime()` matches.

TC-5. MTIME 32-bit half-word R/W

- **Spec:** F7
- **Stimulus:**
 - `write32(mtime_addr() + 0, 0x11223344)`.

- `write32(mtime_addr() + 4, 0x55667788)`.
- Read each half back independently.
- **Expected:** low half = `0x11223344`, high half = `0x55667788`, `dbg_mtime() == 0x5566_7788_1122_3344`.

TC-6. Firmware-style MTIME read sequence

- **Spec:** F8
- **Setup:** `dbg_set_mtime(0x4000_0000_1234_5678)`.
- **Stimulus:** Call the helper `fw_read_mtime(drv)` which mirrors the rollover-safe loop (`hi`, `lo`, `hi`-again `retry`) from `__metal_clint0_mtime_get`.
- **Expected:** returned value equals `0x4000_0000_1234_5678`.

TC-7. MTIMECMP 64-bit R/W per hart

- **Spec:** F9
- **Stimulus** (per hart): `write64(mtimecmp_addr(h), 0xAAAA_BBBB_CCCC_DDDD)`, then `read64(mtimecmp_addr(h))`.
- **Expected:** read returns the written pattern; `dbg_mtimecmp(h)` matches.

TC-8. Firmware-style MTIMECMP set sequence

- **Spec:** F10
- **Setup:** `cmp_target = 0x0000_0001_0000_1000`.
- **Stimulus:** `fw_set_mtimecmp(drv, 0, cmp_target)` — writes `hi=0xFFFFFFFF`, then `lo`, then `hi`.
- **Expected:** `dbg_mtimecmp(0) == cmp_target`.

TC-9. MTIP comparator at -1 / == / +1 boundary

- **Spec:** F11
- **Setup:** `pulse_reset()`; for every hart `write64(mtimecmp_addr(h), 100)`.
- **Stimulus:** `set_mtime_and_settle(99)` → expect all `mtip_sig[h] == false`.
`set_mtime_and_settle(100)` → all `mtip_sig[h] == true`. `set_mtime_and_settle(101)` → all `mtip_sig[h] == true`.
- **Notes:** Confirms the comparator is unsigned and uses $\geq$ (not $>$).

TC-10. MTIP cleared by raising MTIMECMP

- **Spec:** F12
- **Setup:** TC-9 left MTIP asserted.
- **Stimulus:** For every hart, `write64(mtimecmp_addr(h), 0xFFFF_FFFF_FFFF_FFFF)`; settle.
- **Expected:** `mtip_sig[h] == false`.

TC-11. Per-hart MTIMECMP independence

- **Spec:** F13
- **Setup:** `pulse_reset()`; `write64(mtimecmp_addr(0), 50)`, `write64(mtimecmp_addr(1), 200)`.
- **Stimulus:** `set_mtime_and_settle(100)`.

- **Expected:** `mtip_sig[0] == true; mtip_sig[1] == false.`

TC-12. MSIP and MTIP independent per hart

- **Spec:** F14
- **Setup:** `pulse_reset()`; configure hart 0 so MTIP fires at 10.
- **Stimulus:**
 - `set_mtime_and_settle(10) → mtip_sig[0] == true, msip_sig[0] == false.`
 - `write32(msip_addr(0), 1) → msip_sig[0] == true, mtip_sig[0] unchanged.`
 - `write64(mtimecmp_addr(0), max) → mtip_sig[0] == false, msip_sig[0] unchanged.`
 - Clear MSIP and confirm both lines low.

TC-13. MTIME advance — TB-controlled (auto-tick disabled)

- **Spec:** F15
- **Setup:** `pulse_reset(); tick_period_ns == 0` (CCI preset).
- **Stimulus:** Loop: `dbg_set_mtime(before + i)` for `i = 1..1024`.
- **Expected:** `dbg_mtime() == before + 1024`.
- **Notes:** When `tick_period_ns > 0` (e.g. SiP variants), the same test exercises the auto-tick code path and asserts `t1 > t0` after a `wait(5 × tick_period_ns)`.

TC-14. transport_dbg back-door read/write

- **Spec:** F16
- **Stimulus:**
 - `transport_dbg` write of `0xCAFE_BABE_F00D_BAAD` to MTIME.
 - `transport_dbg` read of MTIME → expect the same value.
- **Expected:** `bytes_returned == 8` for each call; `dbg_mtime()` reflects the write.

TC-15. Negative tests — window, alignment, width

- **Spec:** F17, F18, F19
- **Stimulus** (via `raw_xfer`):
 - Read at `0x20000` (out-of-window) → `TLM_ADDRESS_ERROR_RESPONSE`.
 - Read 4 B at `mtime_addr() + 1` (misaligned) → `TLM_BURST_ERROR_RESPONSE`.
 - Read 8 B at `mtime_addr() + 4` (misaligned) → `TLM_BURST_ERROR_RESPONSE`.
 - Read 3 B at `mtime_addr()` (wrong width) → `TLM_BURST_ERROR_RESPONSE`.
 - Read 8 B at `msip_addr(0)` (MSIP is 4 B only) → `TLM_BURST_ERROR_RESPONSE`.

TC-16. CCI introspection — discovery, mutation, immutability

- **Spec:** F20, F21, F22, F23
- **Setup:** `cci_get_broker()` from inside the SystemC hierarchy.
- **Stimulus:**
 - Lookup `tb.clint.num_harts`, `tb.clint.tick_period_ns`, `tb.clint.access_delay_ns` by name.
 - Verify the typed value of `num_harts` matches `num_harts()`.

- Verify the description string is non-empty.
- Mutate `access_delay_ns` to `2 × old`; issue a transaction; read it back via the broker and confirm the new value.
- Attempt to mutate `num_harts` (immutable); confirm value is unchanged afterwards.
- **Expected:** All handles are valid; mutable param accepts the change; immutable param's value remains the original.

TC-17. `dump_state` contains expected fields

- **Spec:** F24
- **Stimulus:** Call `dut.dump_state(oss)`.
- **Expected:** The captured string contains the substrings `"CLINT state"`, `"mtime"`, `"hart[0]"`.

TC-18. MTIMECMP 32-bit half-word READ (low + high halves)

- **Spec:** F26
- **Setup:** `pulse_reset()`; for every hart write `MTIMECMP[h] = 0xDEAD_BEEF_CAFE_F00D` via `write64`.
- **Stimulus** (per hart):
 - `lo = read32(mtimecmp_addr(h) + 0)`.
 - `hi = read32(mtimecmp_addr(h) + 4)`.
- **Expected:** `lo == 0xCAFE_F00D`, `hi == 0xDEAD_BEEF`, `(hi << 32) | lo == MTIMECMP[h]`.
- **Notes:** TC-7 covers the 64-bit read; TC-8 covers the firmware 32-bit write sequence. TC-18 closes the remaining 32-bit half-word *read* corner exercised by RV32 firmware that snapshots MTIMECMP.

TC-19. In-window reserved address is RAZ/WI

- **Spec:** F27
- **Setup:** After reset.
- **Stimulus:** Using `raw_xfer`:
 - 4 B write of `0xDEAD` to `0x0100` (in window, between MSIP and MTIMECMP regions).
 - 4 B read of `0x0100`.
- **Expected:** Both transactions return `TLM_OK_RESPONSE`; the read returns `0` (RAZ).
- **Notes:** Confirms the `reg_read` / `reg_write` cascade fall-through path (RAZ/WI) is hit by an explicit assertion rather than implicit dead code.

TC-20. Streaming-width mismatch → `TLM_BURST_ERROR_RESPONSE`

- **Spec:** F28
- **Setup:** After reset.
- **Stimulus** (`raw_xfer_sw`):
 - 4 B read at `mtime_addr()` with `streaming_width = 1`.
 - 8 B write at `mtime_addr()` with `streaming_width = 4`.
- **Expected:** Both return `TLM_BURST_ERROR_RESPONSE`.
- **Notes:** Resolves negative-test catalogue item N15.

TC-21. Byte-enable error and unknown-command error

- **Spec:** F29, F30
- **Setup:** After reset.
- **Stimulus:**
 - `raw_xfer_be(TLM_READ_COMMAND, mtime_addr(), 4, &scratch, &be, 1)` — non-null byte-enable pointer.
 - `raw_xfer(TLM_IGNORE_COMMAND, mtime_addr(), 4, &scratch)`.
- **Expected:**
 - First call returns `TLM_BYTE_ENABLE_ERROR_RESPONSE`.
 - Second call returns `TLM_COMMAND_ERROR_RESPONSE`.
- **Notes:** Resolves negative-test catalogue items N10 and N11.

TC-22. `transport_dbg` rejects invalid args

- **Spec:** F31
- **Setup:** After reset; bypass `b_transport` by calling `drv.sock->transport_dbg(gp)` directly.
- **Stimulus:** Three failing `transport_dbg` payloads:
 - `data_length = 3` (not 4 or 8) at `mtime_addr()`.
 - `data_length = 4` at `mtime_addr() + 1` (misaligned).
 - `data_length = 4` at `WINDOW_SIZE + 0x100` (out of window).
- **Expected:** Every call returns `0` (bytes serviced).
- **Notes:** Mirrors the same alignment / window / width rules that `b_transport` enforces, but on the debug back-door path.

Auto-tick test cases (`clint_tick_tb.cpp`)

The auto-tick test bench is a separate binary because it forces `tick_period_ns = 50 ns` (the main TB sets `0` for determinism). It uses one hart and shares the same `EXPECT_*` macros.

TA-1. Constructor schedules first tick

- **Spec:** F32 (sub-rule: constructor arms `tick_event_`)
- **Setup:** Bring `rst_n` high; `pulse_reset()`; verify `dbg_mtime() == 0`.
- **Stimulus:** `wait(period_ns * 10 + 5 ns)`.
- **Expected:** `dbg_mtime() >= 9` (at least 9 of the 10 ticks landed).
- **Notes:** Covers `src/clint.cpp` lines 121-122 and 168-179 that the deterministic TB cannot reach.

TA-2. Reset cancels pending tick and re-arms it

- **Spec:** F32 (sub-rule: `reset_proc` cancel + re-arm)
- **Setup:** From TA-1's post-tick state, issue another `pulse_reset()`; verify `MTIME` returns to 0.
- **Stimulus:**
 - `wait(period_ns * 0.4)` — first post-reset tick must not yet have fired; expect `dbg_mtime() == 0`.
 - `wait(period_ns * 0.7)` — first tick must now have fired; expect `dbg_mtime() >= 1`.

- **Expected:** MTIME stays at 0 for the first half-period, then increments. No spurious tick from the pre-reset schedule.
- **Notes:** Covers `src/clint.cpp` lines 151-154 (`tick_event_.cancel()` followed by `tick_event_.notify(tick_period_)`).

TA-3. Auto-tick raises MTIP when $\text{MTIME} \geq \text{MTIMECMP}$

- **Spec:** F32 (sub-rule: tick $\rightarrow$ recompute $\rightarrow$ output)
- **Setup:** `pulse_reset(); write64(mtimecmp_addr(0), 5)`.
- **Stimulus:** `wait(period_ns * (5 + 1.5)); one delta settle()`.
- **Expected:** `dbg_mtime() >= 5`, `dbg_mtip(0) == true`, `mtip_sig[0].read() == true`.
- **Notes:** Closes the gap between TC-9 (TB-driven boundary check) and a fully autonomous tick-driven assertion.

9. Coverage plan

9.1 Functional coverage (instrumentation roadmap)

The current test bench is directed; coverage instrumentation is planned but not yet plumbed into the project's coverage harness. Hooks expected in follow-up commits:

Bin / cross	Implementation hook
Per-register R / W coverage	Increment counters at <code>reg_read</code> / <code>reg_write</code> exit
MTIME tick count	Counter in <code>tick_method</code>
MTIP[h] rising / falling transitions	Counters in <code>output_method</code>
MSIP[h] rising / falling transitions	Counters in <code>output_method</code>
Cross: $(\text{MTIME} - \text{MTIMECMP}) \in \{<0, ==0, >0\}$ per hart	Sample in <code>output_method</code> after each recompute
Cross: (write half $\in \{\text{low}, \text{high}\}$, register class)	Sample in <code>reg_write</code>
Reserved-region accesses	Counter in <code>reg_read</code> / <code>reg_write</code> reserved branch

9.2 Code coverage targets

When integrated with `gcov` / `llvm-cov`, the targets are:

Metric	Target
Line coverage	$\geq 95\%$
Branch coverage	$\geq 90\%$

Unstable lines (e.g. `SC_REPORT_FATAL` paths in the constructor) are explicitly excluded with `// LCOV_EXCL_LINE` comments.

9.3 Coverage closure plan

A test is considered insufficient if any spec requirement in §7 is covered by zero tests, or any covergroup bin in §9.1 has count 0 after the regression. Closure is reached when:

- All spec rows (F1..F32) have at least one passing test (F25 covered structurally; see §7).
- All covergroup bins are hit at least once.
- Negative-test catalogue (§10) is fully exercised.
- Code coverage targets in §9.2 are met.

10. Negative-test catalogue

#	Negative scenario	Expected response	Covered by
N1	Misaligned 4 B read (e.g. <code>addr = mtime+1</code>)	<code>TLM_BURST_ERROR_RESPONSE</code>	TC-15
N2	Misaligned 4 B write	<code>TLM_BURST_ERROR_RESPONSE</code>	TC-15 (symmetric path)
N3	Misaligned 8 B read (<code>addr & 7 ≠ 0</code>)	<code>TLM_BURST_ERROR_RESPONSE</code>	TC-15
N4	Wrong access width (e.g. <code>length = 3</code>)	<code>TLM_BURST_ERROR_RESPONSE</code>	TC-15
N5	8 B access on MSIP (4 B only register)	<code>TLM_BURST_ERROR_RESPONSE</code>	TC-15
N6	Out-of-window access (<code>addr ≥ 0x10000</code>)	<code>TLM_ADDRESS_ERROR_RESPONSE</code>	TC-15
N7	MSIP write of <code>0xFFFFFFFF</code> — bits[31:1] dropped	RAZ/WI	TC-2
N8	Reserved address inside window	RAZ/WI	TC-19
N9	Mutate immutable CCI param <code>num_harts</code>	CCI prints <code>SC_ERROR</code> ; value unchanged	TC-16
N10	<code>TLM_IGNORE_COMMAND</code>	<code>TLM_COMMAND_ERROR_RESPONSE</code>	TC-21
N11	Byte-enable supplied	<code>TLM_BYTE_ENABLE_ERROR_RESPONSE</code>	TC-21
N12	Construction with <code>num_harts = 0</code> or <code>> 4095</code>	<code>SC_REPORT_FATAL</code> at elaboration	Future (planned)
N13	Construction with <code>tick_period_ns < 0</code>	<code>SC_REPORT_FATAL</code> at elaboration	Future (planned)
N14	DMI request	<code>dmi_allowed = false</code> (no callback registered)	Implicit (callback never registered)
N15	Streaming-width $\neq$ data-length	<code>TLM_BURST_ERROR_RESPONSE</code>	TC-20
N16	<code>transport_dbg</code> invalid length / misaligned / out-of-window	<code>transport_dbg</code> returns 0	TC-22
N17	Auto-tick must not fire while in reset	MTIME stays at 0; tick re-armed on rst de-assert	TA-2

Items marked `Future (planned)` are listed in §13.

11. Regression workflow

11.1 Local regression

```
peripherals/clint/run_tests.sh
```

The script:

1. Auto-locates SystemC (`SYSTEMC_HOME` or Homebrew default) and CCI (`CCI_HOME` or `/Users/pdroy/cci`).
2. Configures CMake (only on first run; subsequent runs are incremental).
3. Builds with `-j<ncpus>`.
4. Runs the two test bench binaries:
 - `peripherals/clint/build/test/clint_tb` (TC-1..TC-22)
 - `peripherals/clint/build/test/clint_tick_tb` (TA-1..TA-3)

11.2 CI regression

```
peripherals/clint/run_tests.sh --ctest
```

`ctest` parses the test bench output and asserts on:

- `PASS_REGULAR_EXPRESSION "ALL TESTS PASSED"` — must match.
- `FAIL_REGULAR_EXPRESSION "FAIL"` — must not match.

The command exits non-zero on either condition.

11.3 Sanitizer / coverage runs

```
peripherals/clint/run_tests.sh --asan      # AddressSanitizer-clean baseline
peripherals/clint/run_tests.sh --coverage  # llvm-cov / gcov line coverage report
```

ASan and coverage builds use isolated build trees (`build_asan/`, `build_cov/`) so they never invalidate the main `build/` cache.

11.4 Triage protocol

On failure:

1. Read the first `FAIL` line from `stderr` — it carries `__FILE__:__LINE__` plus the expected/actual values.
 2. Optionally re-run with `--clean` to rule out a stale build.
 3. Add `dut.dump_state(std::cerr)` adjacent to the failing assertion to capture full CLINT state.
 4. If the failure reproduces under a single test case but not in isolation, suspect cross-test state leakage — most likely an unreset `mtimecmp[h]` from a prior block. Inserting `pulse_reset()` at the start of the failing block usually clears it.
-

12. Pass / fail criteria & exit conditions

Outcome	Criterion
PASS	The test bench prints <code>ALL TESTS PASSED</code> and exits with status <code>0</code> .
FAIL	Any <code>EXPECT_*</code> macro reports <code>FAIL</code> , or the bench prints any <code>FAIL</code> line, or it exits with non-zero status.
BUILD FAIL	CMake configuration or compilation error — counts as <code>FAIL</code> for CI.
TIMEOUT (CI only)	Bench has not completed within 60 s — counts as <code>FAIL</code> (current actual: < 1 s).

A green CI run requires every test case TC-1..TC-22 (`clint_tb`) and TA-1..TA-3 (`clint_tick_tb`) to pass on every supported (SystemC version × CCI version × compiler) matrix entry.

13. Future tests & open work

Items not yet covered by `clint_tb.cpp` but planned:

Item	Priority	Notes
Construction-time <code>SC_REPORT_FATAL</code> for bad <code>num_harts</code> / <code>tick_period_ns</code>	Low	Use <code>sc_report_handler::set_actions(...)</code> to capture
DMI request rejection	Low	Verify the absence of a <code>get_direct_mem_ptr</code> handler
Cross-hart IPI scenario (hart-A sets MSIP[B], hart-B clears, hart-B traps)	Medium	Requires Whisper integration
Coverage instrumentation for transitions and cross bins	Medium	Coverage plumbing (see §9.1)
Multi-CLINT instantiation (e.g. one per CPU cluster in multi-cluster SiP)	Low	Build a TB with two <code>clint</code> modules and disjoint harts
Random-stimulus fuzzer driving register accesses + tick events	Medium	Bound-checked random sequence; compare against reference model
Whisper+CLINT system test	High	Boot SMC firmware against the model through the fabric
Parity / lockstep safety extensions (<code>tt-oca-hw.pdf</code> §16)	Low	Pending the safety-extension wrapper

Items previously listed here that have now been added to the active suite: `TLM_IGNORE_COMMAND` (→ TC-21), byte-enable rejection (→ TC-21), streaming-width mismatch (→ TC-20), `transport_dbg` invalid-args (→ TC-22), in-window hole RAZ/WI (→ TC-19), MTIMECMP 32-bit half-word read (→ TC-18), and the auto-tick path (→ TA-1, TA-2, TA-3).

14. Test results — current baseline

Test	Status
TC-1 reset clears MTIME / MSIP / MTIP; MTIMECMP=max	PASS
TC-2 MSIP R/W truncates to bit[0]; msip_o tracks reg	PASS
TC-3 MSIP per-hart isolation	PASS
TC-4 MTIME 64-bit R/W	PASS
TC-5 MTIME 32-bit half-word R/W	PASS
TC-6 firmware-style MTIME read sequence	PASS
TC-7 MTIMECMP 64-bit R/W per hart	PASS
TC-8 firmware-style MTIMECMP set sequence	PASS
TC-9 MTIP comparator at -1 / == / +1 boundary	PASS
TC-10 MTIP cleared by raising MTIMECMP	PASS
TC-11 per-hart MTIMECMP independence	PASS
TC-12 MSIP and MTIP independent per hart	PASS
TC-13 synthetic MTIME advance (auto-tick disabled)	PASS
TC-14 transport_dbg back-door read/write	PASS
TC-15 negative tests: window, alignment, width	PASS
TC-16 CCI: discovery, introspection, mutation, immutability	PASS
TC-17 dump_state contains expected fields	PASS
TC-18 MTIMECMP 32-bit half-word READ (low + high)	PASS
TC-19 in-window hole is RAZ/WI	PASS
TC-20 streaming-width mismatch → BURST_ERROR	PASS
TC-21 byte-enable error and unknown-command error	PASS
TC-22 transport_dbg invalid-args paths return 0	PASS
TA-1 constructor schedules first tick (auto-tick)	PASS
TA-2 reset cancels pending tick; counting restarts	PASS
TA-3 auto-tick raises MTIP when MTIME ≥ MTIMECMP	PASS

Result: 22 / 22 (clint_tb) + 3 / 3 (clint_tick_tb) PASS — both binaries print ALL TESTS PASSED

Environment: SystemC 3.0.2 Accellera, CCI 1.0.2, Apple Clang 17, macOS 24.3.0 **Wall-clock:** < 1 second total on Apple Silicon (both binaries combined). **ASan:** clean (./run_tests.sh --asan reports no memory errors). **Coverage (src/clint.cpp):** line ≥ 96 %, function 100 %, region ≥ 89 % (./run_tests.sh --coverage merges profiles from both binaries; see 02_CLINT_LowLevel_Design.md and the generated HTML report for the per-line breakdown).

End of document.